BFS Templates — Tree

Core structure: `Queue<TreeNode> queue = new ArrayDeque<>()`.
The only decision: do you need to know **which level** a node is on?

The Two Templates

```
// 1. LEVEL-AWARE PROCESSING — snapshot size to make per-level decisions
// MENTAL MODEL: the queue holds exactly one full level; freeze its size, then drain just that many.
// WHEN: "level by level", "per row", "rightmost/leftmost in level", "min depth"
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();       // ← snapshot: all nodes currently in queue = this level
    level++;
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();
        // process node (know: level, i==0 → first, i==size-1 → last)
        if (node.left  != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}

// 2. WITHOUT LEVEL TRACKING — simple node-by-node
// MENTAL MODEL: just visit every node in BFS order; you don't care which level it's on.
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
while (!queue.isEmpty()) {
    TreeNode node = queue.poll();
    // process node
    if (node.left  != null) queue.offer(node.left);
    if (node.right != null) queue.offer(node.right);
}
```

Almost all tree BFS problems use **Template 1** — you nearly always need `size` to know when a level ends.

When to Use — Signal → Pattern

Problem signal phrase	Pattern / group
"level order", "values per level", "each row"	Group 1 — collect entire level
"average / max / sum of each level"	Group 2 — aggregate per level
"right side view" (last in level), "bottom-left" (first in level)	Group 2 — record boundary node (`i==size-1` / `i==0`)
"minimum depth", "first level that satisfies X"	Group 3 — early return on condition
"connect next pointers at the same level"	Group 4 — wire nodes within a level
"same depth, different parent" (cousins), per-node metadata	Group 5 — track per-node metadata
"maximum width of a level" (counting null gaps)	Group 6 — position indexing
no level info needed, just visit every node	Template 2 — node-by-node

The One Rule — Snapshot Level Size First

```
Always snapshot the level size BEFORE the inner for loop:
    int size = queue.size();
    for (int i = 0; i < size; i++) { ... }

Without the snapshot, newly added children mix with the current level
and i == size-1 / i == 0 checks become meaningless.
```

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
102	Binary Tree Level Order Traversal	Return all node values level by level, left to right.		O(n)	O(n)	Standard
107	Binary Tree Level Order Traversal II	Same as #102 but return levels from bottom to top.		O(n)	O(n)	Standard
103	Binary Tree Zigzag Level Order Traversal	Level 1 left-to-right, level 2 right-to-left, alternating.		O(n)	O(n)	Standard
637	Average of Levels in Binary Tree	Return the average value of nodes at each level.		O(n)	O(n)	Standard
199	Binary Tree Right Side View	Return the value of the rightmost node at each level.		O(n)	O(n)	Standard
513	Find Bottom Left Tree Value	Return the leftmost value in the last level (deepest row).		O(n)	O(n)	Standard
515	Find Largest Value in Each Tree Row	Return the maximum value found at each level.		O(n)	O(n)	Standard
1161	Maximum Level Sum of a Binary Tree	Return the smallest level number (1-indexed) whose node sum is maximum.		O(n)	O(n)	Standard
111	Minimum Depth of Binary Tree	Minimum number of nodes from root to the nearest leaf.	BFS reaches nodes in depth order, so the first leaf it dequeues is necessarily the shallowest.	O(n)	O(n)	Standard
116	Populating Next Right Pointers in Each Node	Connect each node's `next` to the node immediately to its right at the same level. Tree is a perfect binary tree.	BFS already visits a level left to right, so just chain each node to the one polled before it.	O(n)	O(n)	Standard
117	Populating Next Right Pointers in Each Node II	Same as #116 but the tree can be any binary tree (not necessarily perfect).		O(n)	O(n)	Standard
993	Cousins in Binary Tree	Two nodes `x` and `y` are cousins if they are at the same depth but have different parents. Return true if `x` and `y` are cousins.	cousins must surface in the *same* BFS level; if only one shows up per level, their depths differ.	O(n)	O(n)	Standard
662	Maximum Width of Binary Tree	Return the maximum width of any level of a binary tree. Width is defined as the length between the leftmost and rightmost non-null nodes, including all the null nodes in between.		O(n)	O(n)	BFS with position tracking. Assign each node a position: root=1, left child of node at pos `p` gets `2*p`, right child gets `2*p+1`. Width of level = `lastPos - firstPos + 1`. Normalize by subtracting `firstPos` of each level to prevent overflow.

Group 1 — Collect Entire Level

#102 Binary Tree Level Order Traversal

Description: Return all node values level by level, left to right.

```java
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#107 Binary Tree Level Order Traversal II

Description: Same as #102 but return levels from bottom to top.
Key: `LinkedList` as result — `addFirst` prepends each level, producing bottom-up order without a final reverse.

```java
class Solution {
    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        LinkedList<List<Integer>> result = new LinkedList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.addFirst(level);    // prepend → bottom-up order
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#103 Binary Tree Zigzag Level Order Traversal

Description: Level 1 left-to-right, level 2 right-to-left, alternating.
Key: `LinkedList<Integer>` per level as a deque — `addLast` for left-to-right, `addFirst` for right-to-left. Flip flag after each level.

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        boolean leftToRight = true;
        while (!queue.isEmpty()) {
            int size = queue.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (leftToRight) {
                    level.addLast(node.val);
                } else {
                    level.addFirst(node.val);
                }
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
            leftToRight = !leftToRight;
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#637 Average of Levels in Binary Tree

Description: Return the average value of nodes at each level.

```
class Solution {
    public List<Double> averageOfLevels(TreeNode root) {
        List<Double> result = new ArrayList<>();
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            double sum = 0;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                sum += node.val;
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(sum / size);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

Group 2 — Record First / Last / Max in Each Level

All share the same outer loop. Only the `if` inside the `for` changes.

#199 Binary Tree Right Side View

Description: Return the value of the rightmost node at each level.
Key: record when `i == size - 1`.

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (i == size - 1) result.add(node.val);   // last in level
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#513 Find Bottom Left Tree Value

Description: Return the leftmost value in the last level (deepest row).
Key: record when `i == 0` every level — after the loop, `bottomLeft` holds the first node of the final level.

```
class Solution {
    public int findBottomLeftValue(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        int bottomLeft = root.val;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (i == 0) bottomLeft = node.val;         // first in level — overwritten each level
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return bottomLeft;
    }
}
```

Time O(n) | **Space** O(n)

#515 Find Largest Value in Each Tree Row

Description: Return the maximum value found at each level.

```java
class Solution {
    public List<Integer> largestValues(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            int max = Integer.MIN_VALUE;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                max = Math.max(max, node.val);
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(max);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#1161 Maximum Level Sum of a Binary Tree

Description: Return the smallest level number (1-indexed) whose node sum is maximum.

```java
class Solution {
    public int maxLevelSum(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        int maxSum = Integer.MIN_VALUE, maxLevel = 1, level = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            level++;
            int sum = 0;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                sum += node.val;
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            if (sum > maxSum) { maxSum = sum; maxLevel = level; }
        }
        return maxLevel;
    }
}
```

Time O(n) | **Space** O(n)

Group 3 — Early Return on Condition

#111 Minimum Depth of Binary Tree

Description: Minimum number of nodes from root to the nearest leaf.
Key: BFS is optimal here (DFS must visit the whole tree). Return `depth` the moment the first leaf is dequeued — that is guaranteed to be the shallowest.
Intuition: BFS reaches nodes in depth order, so the first leaf it dequeues is necessarily the shallowest.

```java
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) return 0;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        int depth = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            depth++;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (node.left == null && node.right == null) return depth;  // first leaf
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return depth;
    }
}
```

Time O(n) | **Space** O(n)

Group 4 — Connect Nodes Within a Level

#116 Populating Next Right Pointers in Each Node

Description: Connect each node's `next` to the node immediately to its right at the same level. Tree is a perfect binary tree.
Key: `previous` tracks the last node seen in the current level — wire `previous.next = node` before advancing. Reset `previous = null` at each level start.
Intuition: BFS already visits a level left to right, so just chain each node to the one polled before it.

```java
class Solution {
    public Node connect(Node root) {
        if (root == null) return null;
        Queue<Node> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            Node previous = null;
            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                if (previous != null) previous.next = node;
                previous = node;
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return root;
    }
}
```

Time O(n) | **Space** O(n)

#117 Populating Next Right Pointers in Each Node II

Description: Same as #116 but the tree can be any binary tree (not necessarily perfect).
Key: the BFS solution is **identical** to #116 — the `size` snapshot handles uneven levels automatically.

```java
class Solution {
    public Node connect(Node root) {
        if (root == null) return null;
        Queue<Node> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            Node previous = null;
            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                if (previous != null) previous.next = node;
                previous = node;
                if (node.left  != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return root;
    }
}
```

Time O(n) | **Space** O(n)

Group 5 — Track Per-Node Metadata

#993 Cousins in Binary Tree

Description: Two nodes `x` and `y` are cousins if they are at the same depth but have different parents. Return true if `x` and `y` are cousins.
Key: for each node polled, check if its children are `x` or `y`. After each full level, if both parents found → check `xParent != yParent`. If only one found → different depths → false.
Intuition: cousins must surface in the *same* BFS level; if only one shows up per level, their depths differ.

```java
class Solution {
    public boolean isCousins(TreeNode root, int x, int y) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            TreeNode xParent = null, yParent = null;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (node.left != null) {
                    queue.offer(node.left);
                    if (node.left.val == x) xParent = node;
                    if (node.left.val == y) yParent = node;
                }
                if (node.right != null) {
                    queue.offer(node.right);
                    if (node.right.val == x) xParent = node;
                    if (node.right.val == y) yParent = node;
                }
            }
            if (xParent != null && yParent != null) return xParent != yParent;
            if (xParent != null || yParent != null) return false;  // one found → different depths
        }
        return false;
    }
}
```

Time O(n) | **Space** O(n)

Side-by-Side: 199 vs 513

Both record one node per level. The only difference is which node:

```
                    #199 Right Side View        #513 Bottom Left Value
Record condition:   i == size - 1 (last)        i == 0 (first)
Update per level:   result.add(node.val)        bottomLeft = node.val  (overwrite)
Return:             List<Integer>               bottomLeft after all levels done
```

Side-by-Side: 102 vs 103 vs 107

All collect every node in every level. Only the insertion order changes:

```
                    #102 Level Order            #103 Zigzag             #107 Bottom-Up
Level container:    ArrayList<Integer>          LinkedList<Integer>     ArrayList<Integer>
Add node:           level.add(val)              addLast/addFirst        level.add(val)
Add to result:      result.add(level)           result.add(level)       result.addFirst(level)
Extra state:        —                           boolean leftToRight     result is LinkedList
```

Pattern Summary

What changes per level	Key variable	Example problems
Collect all values	`List<Integer> level`	102, 107, 103
Compute aggregate	`int sum` / `int max`	637, 515, 1161
Record boundary node	check `i == 0` or `i == size-1`	199, 513
Return on first match	`return depth`	111
Wire nodes together	`Node previous`	116, 117
Track parent metadata	`TreeNode xParent, yParent`	993

The One Rule (recap)

Snapshot the level size **before** the inner `for` loop — see "The One Rule" at the top of this file for the full explanation.

Group 6 — Position Indexing

#662 Maximum Width of Binary Tree

Description: Return the maximum width of any level of a binary tree. Width is defined as the length between the leftmost and rightmost non-null nodes, including all the null nodes in between.

Variation: BFS with position tracking. Assign each node a position: root=1, left child of node at pos `p` gets `2*p`, right child gets `2*p+1`. Width of level = `lastPos - firstPos + 1`. Normalize by subtracting `firstPos` of each level to prevent overflow.

```java
class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        if (root == null) return 0;
        int maxWidth = 0;
        List<TreeNode> nodes = new ArrayList<>();
        List<Long> positions = new ArrayList<>();
        nodes.add(root);
        positions.add(1L);
        while (!nodes.isEmpty()) {
            int size = nodes.size();
            long start = positions.get(0);
            maxWidth = (int) Math.max(maxWidth, positions.get(size-1) - start + 1);
            List<TreeNode> nextNodes = new ArrayList<>();
            List<Long> nextPositions = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = nodes.get(i);
                long pos = positions.get(i) - start;     // ← VARIATION: normalize to prevent overflow
                if (node.left != null) {
                    nextNodes.add(node.left);
                    nextPositions.add(2 * pos);           // ← VARIATION: left child = 2*pos
                }
                if (node.right != null) {
                    nextNodes.add(node.right);
                    nextPositions.add(2 * pos + 1);       // ← VARIATION: right child = 2*pos+1
                }
            }
            nodes = nextNodes;
            positions = nextPositions;
        }
        return maxWidth;
    }
}
```

Time O(n) | **Space** O(n)